

Laporan Praktikum Kontrol Cerdas

Nama : Gunawan Wibisono
NIM : 224308083
Kelas : TKA – 7D
Akun *Github* (Tautan) : <https://github.com/gunawanwibisono83>
Student Lab Assistant : -

1. Judul Percobaan

Reinforcement Learning for Autonomous Control

2. Tujuan Percobaan

Tujuan dilakukannya percobaan ini yaitu :

- a. Mahasiswa dapat memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali.
- b. Mahasiswa dapat mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
- c. Mahasiswa dapat menggunakan *OpenAI* sebagai simulasi lingkungan untuk pelatihan RL.
- d. Mahasiswa dapat melatih dan menguji RL untuk mengontrol lingkungan secara otonom.

3. Landasan Teori

Machine learning dapat didefinisikan sebagai aplikasi komputer dan algoritma matematika yang diadopsi dengan cara pembelajaran yang berasal dari data dan menghasilkan prediksi di masa yang akan datang (Goldberg & Holland, 1988). *Machine learning* juga dapat diartikan sebuah *computer* yang memiliki kemampuan belajar tanpa diprogram secara eksplisit. Program tersebut memanfaatkan data untuk membangun model dan mengambil keputusan berdasarkan model yang telah dibangun. *Machine learning* dapat dikelompokkan berdasarkan bagaimana cara belajar sehingga dapat melakukan

tugasnya. Pembagian *machine learning* berdasarkan cara belajarnya dibagi menjadi tiga yaitu *Supervised Learning*, *Unsupervised Learning*, dan *Reinforcement Learning* (Daqiqil, 2021).

Reinforcement Learning (RL) adalah salah satu metode *machine learning* yang berfokus pada bagaimana agen harus mengambil tindakan dalam suatu lingkungan untuk memaksimalkan total hadiah yang diperolehnya. Berbeda dengan *supervised learning* yang membutuhkan data berlabel untuk pelatihan, RL tidak memerlukan pasangan input-output eksplisit. Sebaliknya, RL menggunakan mekanisme *trial and error*, di mana agen belajar dari konsekuensi tindakannya sendiri melalui umpan balik berbentuk *reward* atau *punishment*. Di sisi lain, *unsupervised learning* tidak menggunakan label dan berfokus pada menemukan pola atau struktur tersembunyi dalam data, seperti *clustering* atau asosiasi. Sementara *supervised learning* bekerja dengan data yang terstruktur dengan baik dan berlabel, *unsupervised learning* mencoba memahami struktur internal data tanpa panduan eksplisit, dan RL berinteraksi dinamis dengan lingkungan untuk belajar secara progresif dari pengalaman. Dengan demikian, RL menawarkan pendekatan unik yang sangat berbeda dari *supervised* dan *unsupervised learning*, dengan aplikasi yang luas dalam bidang seperti robotika, permainan, dan sistem rekomendasi (Mehmood, 2024).

Deep Q learning merupakan suatu metode yang ide dasarnya dari metode *Q learning* yang dikombinasikan dengan *deep learning*. *Deep Q learning* menghasilkan performa yang baik dan sangat relevan dengan perubahan *environment*. Apabila ada perubahan *environment* menjadi kompleks maka isi *Q-table* tidak menjadi efektif lagi karena harus disesuaikan dengan kondisi *environment*. Hal inilah yang menjadi alasan harus menggunakan model arsitektur jaringan saraf/*neural network* untuk menggantikan data tabel. DQL menyimpan pengetahuan dalam bentuk jaringan saraf. Input *neuron* dari jaringan digunakan sebagai *state* dan output neuron *action* dengan nilai Q maksimal (Arifin dkk., 2023). *Gymnasium* adalah *library open source* yang merupakan *fork* dari *OpenAI's Gym*, yang menawarkan standarisasi API untuk *environment reinforcement learning*. (Alwi, 2025).

4. Analisis

Dalam praktikum ini, saya menerapkan salah satu cabang dari *machine learning*, yaitu *Reinforcement Learning* (RL) dengan pendekatan *Deep Q-Network* (DQN) untuk mengendalikan lingkungan simulasi *CartPole-v1* yang disediakan oleh *Gymnasium*. Lingkungan ini berupa sebuah kereta dengan tiang tegak yang harus dijaga keseimbangannya dengan cara memberikan aksi ke kiri atau ke kanan. Agen RL dilatih untuk memaksimalkan total *reward*, di mana *reward* diberikan setiap kali tiang tetap tegak, dan episode berakhir ketika tiang jatuh atau kereta keluar dari batas lintasan.

Proses kerja sistem ini terbagi menjadi beberapa tahapan. Pertama, agen didefinisikan dengan arsitektur jaringan saraf sederhana menggunakan *Keras Sequential Model*, yang memprediksi nilai Q untuk setiap aksi. Kedua, agen menyimpan pengalaman interaksi dalam *replay memory*, yang kemudian diambil secara acak dalam *batch* kecil untuk melatih jaringan sehingga mencegah *overfitting* dan meningkatkan stabilitas pembelajaran. Ketiga, algoritma *epsilon-greedy* digunakan untuk menyeimbangkan eksplorasi dan eksploitasi. Pada awal *training*, agen banyak melakukan eksplorasi (*random action*), kemudian nilai epsilon secara bertahap menurun agar agen lebih fokus pada strategi terbaik yang sudah dipelajari.

Pada percobaan, proses *training* dilakukan selama 1000 episode dengan *batch size* 64. Grafik *reward* tiap episode memperlihatkan bahwa pada awal *training* agen hanya mampu mencapai skor rendah karena aksi masih acak. Namun, seiring bertambahnya episode, agen mulai belajar menjaga keseimbangan dengan lebih lama, sehingga stabilitas performa naik cukup tinggi. Hal ini disebabkan oleh jumlah episode yang banyak, sehingga model hampir benar-benar mencapai performa optimal. Model yang telah dilatih kemudian disimpan dalam format *.h5* untuk digunakan pada pengujian berikutnya.

Secara keseluruhan, model DQN yang digunakan pada percobaan ini sudah mampu belajar pengendalian *CartPole* dengan performa yang cukup optimal. Akurasi pengendalian masih bisa ditingkatkan dengan menambah jumlah episode *training*, menyesuaikan struktur jaringan agar lebih kompleks, atau

menggunakan teknik lanjutan seperti *target network* dan *double DQN*. Berdasarkan beberapa literasi, perubahan parameter berpengaruh besar terhadap kinerja agen. Nilai *gamma* menentukan apakah agen lebih fokus pada hasil jangka pendek atau jangka panjang. Parameter *epsilon* digunakan untuk mengatur keseimbangan antara mencoba hal baru (eksplorasi) dan menggunakan pengalaman terbaik yang sudah ada (eksploitasi). Sementara itu, *learning rate* berfungsi sebagai kecepatan belajar agen. Dengan pengaturan yang seimbang, agen dapat beradaptasi dan meningkatkan performanya secara bertahap.

Tantangan utama selama pelatihan adalah lamanya waktu training, karena dibutuhkan banyak episode agar agen benar-benar stabil. Selain itu, performa agen di awal juga sering tidak konsisten karena masih banyak melakukan eksplorasi. Keterbatasan perangkat keras (*hardware*) juga menjadi kendala karena membuat proses *training* lebih lama sehingga sulit mencoba banyak variasi parameter sekaligus.

5. Diskusi

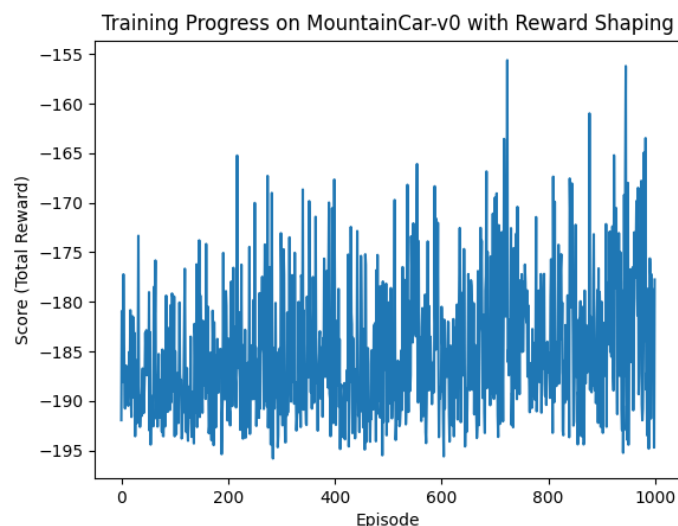
Dalam *supervised learning*, sistem belajar dari data yang sudah diberi label, sehingga hasil yang benar sudah diketahui. Sementara itu, dalam *reinforcement learning*, agen tidak memiliki label melainkan belajar dengan berinteraksi di lingkungan sekitar dan menerima masukan berupa *reward* atau *penalty*. Oleh karena itu, *reinforcement learning* lebih tepat digunakan untuk menyelesaikan masalah yang membutuhkan pengambilan keputusan secara bertahap.

Eksplorasi adalah proses ketika agen mencoba tindakan baru secara acak untuk menemukan cara yang lebih baik, sedangkan eksploitasi adalah proses ketika agen menggunakan cara yang sudah diketahui untuk mendapatkan hasil yang baik. Jika agen terlalu banyak melakukan eksplorasi, proses belajar jadi lebih lama karena terlalu sering mencoba hal yang tidak efektif. Di sisi lain, jika agen terlalu cepat beralih ke eksploitasi, agen bisa terjebak pada cara yang kurang baik karena tidak mencoba pilihan lain. Untuk mengatasi masalah ini, dapat digunakan strategi dengan agen mulai dengan tingkat eksplorasi yang tinggi, lalu secara perlahan mengurangnya sehingga semakin lama agen lebih fokus pada cara yang terbaik yang sudah dipelajari.

6. Assignment

Setelah melakukan pelatihan agen pada *environment* CartPole-v1, praktikum dilanjutkan dengan menguji algoritma DQN pada *environment* lain yaitu MountainCar-v0. Pada CartPole-v1, tujuan agen adalah menjaga tiang tetap tegak di atas kereta. *Reward* diberikan positif (+1) setiap langkah selama tiang masih seimbang, dan episode berakhir ketika tiang jatuh atau kereta keluar dari batas. Pola *reward* yang lebih jelas ini membuat agen relatif lebih mudah belajar karena setiap langkah yang benar langsung menghasilkan umpan balik positif.

Berbeda dengan itu, MountainCar-v0 memiliki tantangan yang lebih besar. Dalam *environment* ini, agen harus menggerakkan mobil agar mencapai puncak bukit di sebelah kanan. *Reward default* yang diberikan adalah -1 setiap langkah, dan hanya jika agen berhasil mencapai puncak maka episode dihentikan dengan *reward* tambahan. Hal ini membuat agen lebih sulit belajar, karena pada awal *training* hampir semua episode berakhir dengan *reward* -200 (batas langkah maksimum). Agen hanya bisa belajar strategi dengan memanfaatkan momentum (bergerak maju-mundur) setelah melalui banyak episode dan dengan bantuan *reward shaping*.



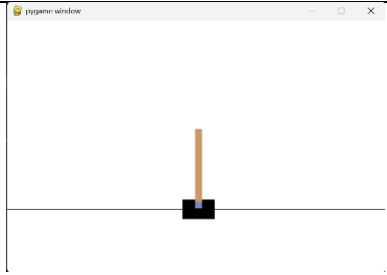
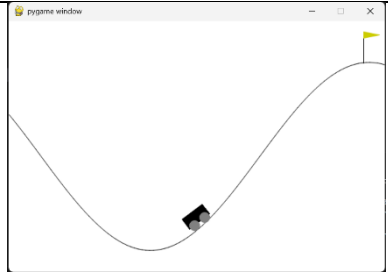
Gambar 1. Grafik skor reward MountainCar-v0

Berdasarkan Gambar 1. Diatas menunjukkan perkembangan *reward* agen DQN pada *environment* MountainCar-v0 dengan penerapan *reward shaping* selama 1000 episode pelatihan. Pada awal *training*, skor rata-rata masih berada

di sekitar (-195) hingga (-200) yang menandakan agen gagal mencapai puncak bukit dan hanya bertahan hingga batas langkah maksimum. Seiring bertambahnya episode, terlihat adanya peningkatan performa, di mana agen mulai memperoleh reward yang lebih baik, meskipun masih fluktuatif dengan kisaran (-175) hingga (-165) pada pertengahan hingga (-138) akhir *training*. Pola ini menunjukkan bahwa agen mulai belajar memanfaatkan momentum untuk bergerak mendekati puncak, namun strategi yang dihasilkan belum stabil sepenuhnya. Secara keseluruhan, *reward shaping* terbukti membantu agen belajar lebih baik dibandingkan *reward* asli, meskipun untuk mencapai hasil optimal masih dibutuhkan jumlah episode lebih banyak serta modifikasi parameter atau metode lanjutan seperti Double DQN.

7. Data dan Output Hasil Pengamatan

No.	Variabel	Hasil Pengamatan	
		CartPole-v1	MountainCar-v0
1.	Jumlah episode <i>training</i>	Proses <i>training</i> CartPole-v1 dilakukan sebanyak 1000 episode	Proses <i>training</i> MountainCar-v0 dilakukan sebanyak 1000 episode
2.	Nilai Awal	<pre>To enable the following instructions: SSE3 SSE4.1 SSE4.2 SSE4.4 SSE4.5 SSE4.7 SSE4.8 SSE4.10 SSE4.11 SSE4.12 SSE4.13 SSE4.14 SSE4.15 SSE4.16 SSE4.17 SSE4.18 SSE4.19 SSE4.20 SSE4.21 SSE4.22 SSE4.23 SSE4.24 SSE4.25 SSE4.26 SSE4.27 SSE4.28 SSE4.29 SSE4.30 SSE4.31 SSE4.32 SSE4.33 SSE4.34 SSE4.35 SSE4.36 SSE4.37 SSE4.38 SSE4.39 SSE4.40 SSE4.41 SSE4.42 SSE4.43 SSE4.44 SSE4.45 SSE4.46 SSE4.47 SSE4.48 SSE4.49 SSE4.50 SSE4.51 SSE4.52 SSE4.53 SSE4.54 SSE4.55 SSE4.56 SSE4.57 SSE4.58 SSE4.59 SSE4.60 SSE4.61 SSE4.62 SSE4.63 SSE4.64 SSE4.65 SSE4.66 SSE4.67 SSE4.68 SSE4.69 SSE4.70 SSE4.71 SSE4.72 SSE4.73 SSE4.74 SSE4.75 SSE4.76 SSE4.77 SSE4.78 SSE4.79 SSE4.80 SSE4.81 SSE4.82 SSE4.83 SSE4.84 SSE4.85 SSE4.86 SSE4.87 SSE4.88 SSE4.89 SSE4.90 SSE4.91 SSE4.92 SSE4.93 SSE4.94 SSE4.95 SSE4.96 SSE4.97 SSE4.98 SSE4.99 SSE5.0 SSE5.1 SSE5.2 SSE5.3 SSE5.4 SSE5.5 SSE5.6 SSE5.7 SSE5.8 SSE5.9 SSE5.10 SSE5.11 SSE5.12 SSE5.13 SSE5.14 SSE5.15 SSE5.16 SSE5.17 SSE5.18 SSE5.19 SSE5.20 SSE5.21 SSE5.22 SSE5.23 SSE5.24 SSE5.25 SSE5.26 SSE5.27 SSE5.28 SSE5.29 SSE5.30 SSE5.31 SSE5.32 SSE5.33 SSE5.34 SSE5.35 SSE5.36 SSE5.37 SSE5.38 SSE5.39 SSE5.40 SSE5.41 SSE5.42 SSE5.43 SSE5.44 SSE5.45 SSE5.46 SSE5.47 SSE5.48 SSE5.49 SSE5.50 SSE5.51 SSE5.52 SSE5.53 SSE5.54 SSE5.55 SSE5.56 SSE5.57 SSE5.58 SSE5.59 SSE5.60 SSE5.61 SSE5.62 SSE5.63 SSE5.64 SSE5.65 SSE5.66 SSE5.67 SSE5.68 SSE5.69 SSE5.70 SSE5.71 SSE5.72 SSE5.73 SSE5.74 SSE5.75 SSE5.76 SSE5.77 SSE5.78 SSE5.79 SSE5.80 SSE5.81 SSE5.82 SSE5.83 SSE5.84 SSE5.85 SSE5.86 SSE5.87 SSE5.88 SSE5.89 SSE5.90 SSE5.91 SSE5.92 SSE5.93 SSE5.94 SSE5.95 SSE5.96 SSE5.97 SSE5.98 SSE5.99 SSE6.0 SSE6.1 SSE6.2 SSE6.3 SSE6.4 SSE6.5 SSE6.6 SSE6.7 SSE6.8 SSE6.9 SSE6.10 SSE6.11 SSE6.12 SSE6.13 SSE6.14 SSE6.15 SSE6.16 SSE6.17 SSE6.18 SSE6.19 SSE6.20 SSE6.21 SSE6.22 SSE6.23 SSE6.24 SSE6.25 SSE6.26 SSE6.27 SSE6.28 SSE6.29 SSE6.30 SSE6.31 SSE6.32 SSE6.33 SSE6.34 SSE6.35 SSE6.36 SSE6.37 SSE6.38 SSE6.39 SSE6.40 SSE6.41 SSE6.42 SSE6.43 SSE6.44 SSE6.45 SSE6.46 SSE6.47 SSE6.48 SSE6.49 SSE6.50 SSE6.51 SSE6.52 SSE6.53 SSE6.54 SSE6.55 SSE6.56 SSE6.57 SSE6.58 SSE6.59 SSE6.60 SSE6.61 SSE6.62 SSE6.63 SSE6.64 SSE6.65 SSE6.66 SSE6.67 SSE6.68 SSE6.69 SSE6.70 SSE6.71 SSE6.72 SSE6.73 SSE6.74 SSE6.75 SSE6.76 SSE6.77 SSE6.78 SSE6.79 SSE6.80 SSE6.81 SSE6.82 SSE6.83 SSE6.84 SSE6.85 SSE6.86 SSE6.87 SSE6.88 SSE6.89 SSE6.90 SSE6.91 SSE6.92 SSE6.93 SSE6.94 SSE6.95 SSE6.96 SSE6.97 SSE6.98 SSE6.99 SSE7.0 SSE7.1 SSE7.2 SSE7.3 SSE7.4 SSE7.5 SSE7.6 SSE7.7 SSE7.8 SSE7.9 SSE7.10 SSE7.11 SSE7.12 SSE7.13 SSE7.14 SSE7.15 SSE7.16 SSE7.17 SSE7.18 SSE7.19 SSE7.20 SSE7.21 SSE7.22 SSE7.23 SSE7.24 SSE7.25 SSE7.26 SSE7.27 SSE7.28 SSE7.29 SSE7.30 SSE7.31 SSE7.32 SSE7.33 SSE7.34 SSE7.35 SSE7.36 SSE7.37 SSE7.38 SSE7.39 SSE7.40 SSE7.41 SSE7.42 SSE7.43 SSE7.44 SSE7.45 SSE7.46 SSE7.47 SSE7.48 SSE7.49 SSE7.50 SSE7.51 SSE7.52 SSE7.53 SSE7.54 SSE7.55 SSE7.56 SSE7.57 SSE7.58 SSE7.59 SSE7.60 SSE7.61 SSE7.62 SSE7.63 SSE7.64 SSE7.65 SSE7.66 SSE7.67 SSE7.68 SSE7.69 SSE7.70 SSE7.71 SSE7.72 SSE7.73 SSE7.74 SSE7.75 SSE7.76 SSE7.77 SSE7.78 SSE7.79 SSE7.80 SSE7.81 SSE7.82 SSE7.83 SSE7.84 SSE7.85 SSE7.86 SSE7.87 SSE7.88 SSE7.89 SSE7.90 SSE7.91 SSE7.92 SSE7.93 SSE7.94 SSE7.95 SSE7.96 SSE7.97 SSE7.98 SSE7.99 SSE8.0 SSE8.1 SSE8.2 SSE8.3 SSE8.4 SSE8.5 SSE8.6 SSE8.7 SSE8.8 SSE8.9 SSE8.10 SSE8.11 SSE8.12 SSE8.13 SSE8.14 SSE8.15 SSE8.16 SSE8.17 SSE8.18 SSE8.19 SSE8.20 SSE8.21 SSE8.22 SSE8.23 SSE8.24 SSE8.25 SSE8.26 SSE8.27 SSE8.28 SSE8.29 SSE8.30 SSE8.31 SSE8.32 SSE8.33 SSE8.34 SSE8.35 SSE8.36 SSE8.37 SSE8.38 SSE8.39 SSE8.40 SSE8.41 SSE8.42 SSE8.43 SSE8.44 SSE8.45 SSE8.46 SSE8.47 SSE8.48 SSE8.49 SSE8.50 SSE8.51 SSE8.52 SSE8.53 SSE8.54 SSE8.55 SSE8.56 SSE8.57 SSE8.58 SSE8.59 SSE8.60 SSE8.61 SSE8.62 SSE8.63 SSE8.64 SSE8.65 SSE8.66 SSE8.67 SSE8.68 SSE8.69 SSE8.70 SSE8.71 SSE8.72 SSE8.73 SSE8.74 SSE8.75 SSE8.76 SSE8.77 SSE8.78 SSE8.79 SSE8.80 SSE8.81 SSE8.82 SSE8.83 SSE8.84 SSE8.85 SSE8.86 SSE8.87 SSE8.88 SSE8.89 SSE8.90 SSE8.91 SSE8.92 SSE8.93 SSE8.94 SSE8.95 SSE8.96 SSE8.97 SSE8.98 SSE8.99 SSE9.0 SSE9.1 SSE9.2 SSE9.3 SSE9.4 SSE9.5 SSE9.6 SSE9.7 SSE9.8 SSE9.9 SSE10.0 SSE10.1 SSE10.2 SSE10.3 SSE10.4 SSE10.5 SSE10.6 SSE10.7 SSE10.8 SSE10.9 SSE10.10 SSE10.11 SSE10.12 SSE10.13 SSE10.14 SSE10.15 SSE10.16 SSE10.17 SSE10.18 SSE10.19 SSE10.20 SSE10.21 SSE10.22 SSE10.23 SSE10.24 SSE10.25 SSE10.26 SSE10.27 SSE10.28 SSE10.29 SSE10.30 SSE10.31 SSE10.32 SSE10.33 SSE10.34 SSE10.35 SSE10.36 SSE10.37 SSE10.38 SSE10.39 SSE10.40 SSE10.41 SSE10.42 SSE10.43 SSE10.44 SSE10.45 SSE10.46 SSE10.47 SSE10.48 SSE10.49 SSE10.50 SSE10.51 SSE10.52 SSE10.53 SSE10.54 SSE10.55 SSE10.56 SSE10.57 SSE10.58 SSE10.59 SSE10.60 SSE10.61 SSE10.62 SSE10.63 SSE10.64 SSE10.65 SSE10.66 SSE10.67 SSE10.68 SSE10.69 SSE10.70 SSE10.71 SSE10.72 SSE10.73 SSE10.74 SSE10.75 SSE10.76 SSE10.77 SSE10.78 SSE10.79 SSE10.80 SSE10.81 SSE10.82 SSE10.83 SSE10.84 SSE10.85 SSE10.86 SSE10.87 SSE10.88 SSE10.89 SSE10.90 SSE10.91 SSE10.92 SSE10.93 SSE10.94 SSE10.95 SSE10.96 SSE10.97 SSE10.98 SSE10.99 SSE11.0 SSE11.1 SSE11.2 SSE11.3 SSE11.4 SSE11.5 SSE11.6 SSE11.7 SSE11.8 SSE11.9 SSE11.10 SSE11.11 SSE11.12 SSE11.13 SSE11.14 SSE11.15 SSE11.16 SSE11.17 SSE11.18 SSE11.19 SSE11.20 SSE11.21 SSE11.22 SSE11.23 SSE11.24 SSE11.25 SSE11.26 SSE11.27 SSE11.28 SSE11.29 SSE11.30 SSE11.31 SSE11.32 SSE11.33 SSE11.34 SSE11.35 SSE11.36 SSE11.37 SSE11.38 SSE11.39 SSE11.40 SSE11.41 SSE11.42 SSE11.43 SSE11.44 SSE11.45 SSE11.46 SSE11.47 SSE11.48 SSE11.49 SSE11.50 SSE11.51 SSE11.52 SSE11.53 SSE11.54 SSE11.55 SSE11.56 SSE11.57 SSE11.58 SSE11.59 SSE11.60 SSE11.61 SSE11.62 SSE11.63 SSE11.64 SSE11.65 SSE11.66 SSE11.67 SSE11.68 SSE11.69 SSE11.70 SSE11.71 SSE11.72 SSE11.73 SSE11.74 SSE11.75 SSE11.76 SSE11.77 SSE11.78 SSE11.79 SSE11.80 SSE11.81 SSE11.82 SSE11.83 SSE11.84 SSE11.85 SSE11.86 SSE11.87 SSE11.88 SSE11.89 SSE11.90 SSE11.91 SSE11.92 SSE11.93 SSE11.94 SSE11.95 SSE11.96 SSE11.97 SSE11.98 SSE11.99 SSE12.0 SSE12.1 SSE12.2 SSE12.3 SSE12.4 SSE12.5 SSE12.6 SSE12.7 SSE12.8 SSE12.9 SSE12.10 SSE12.11 SSE12.12 SSE12.13 SSE12.14 SSE12.15 SSE12.16 SSE12.17 SSE12.18 SSE12.19 SSE12.20 SSE12.21 SSE12.22 SSE12.23 SSE12.24 SSE12.25 SSE12.26 SSE12.27 SSE12.28 SSE12.29 SSE12.30 SSE12.31 SSE12.32 SSE12.33 SSE12.34 SSE12.35 SSE12.36 SSE12.37 SSE12.38 SSE12.39 SSE12.40 SSE12.41 SSE</pre>	

5.	Kondisi Akhir		
6.	Kondisi Berhasil	Agen dapat mempertahankan tiang dengan skor yang tinggi.	Agen dapat menjalankan mobil sampai puncak bukit dengan skor yang tinggi.
7.	Kinerja agen	Dengan proses <i>training</i> yang dilakukan sebanyak 1000 episode, agen dapat mempertahankan tiang dengan baik sehingga mendapatkan nilai yang tinggi yaitu 499.	Dengan proses <i>training</i> yang dilakukan sebanyak 1000 episode, agen masih mengalami kesusahan dalam menjalankan mobil sampai puncak sehingga nilai yang didapatkan relatif rendah sebesar -138.

8. Kesimpulan

Berdasarkan percobaan yang telah dilakukan, maka dapat diambil kesimpulan sebagai berikut:

- Mahasiswa telah berhasil mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN) untuk mengendalikan lingkungan simulasi *CartPole-v1* dan *MountainCar-v0* yang disediakan oleh *Gymnasium*.
- Hasil pengujian menunjukkan bahwa agen mampu meningkatkan performanya secara bertahap, meskipun pada percobaan awal skor masih fluktuatif akibat keterbatasan jumlah episode *training*.
- Pada *CartPole-v1*, agen mampu belajar dengan cepat dan menunjukkan peningkatan performa yang stabil. *Reward* meningkat seiring bertambahnya episode, menandakan agen berhasil menemukan strategi yang optimal.
- Pada *MountainCar-v0*, agen masih kesulitan mencapai puncak bukit sehingga *reward* cenderung pada nilai rendah. Hal ini menunjukkan performa agen belum optimal.
- Hasil praktikum menunjukkan bahwa *CartPole-v1* lebih mudah dipahami agen daripada *MountainCar-v0*, hal ini disebabkan oleh tingkat kesulitan lingkungan yang lebih rendah serta pemberian *reward* yang lebih informatif.

9. Saran

Untuk meningkatkan kinerja, dapat dilakukan dengan menambah jumlah episode menjadi lebih banyak saat proses *training*, lalu memodifikasi pengaturan parameter (gamma, epsilon, learning rate), serta penggunaan metode lanjutan seperti *Double DQN* atau *Prioritized Experience Replay*.

10. Daftar Pustaka

- Alwi, A. J. (2025). *IMPLEMENTASI UNITY-GYMNASIUM PADA ENVIRONMENT REINFORCEMENT LEARNING BERBASIS UNITY* [Tugas Akhir, Universitas Islam Indonesia]. <https://dspace.uui.ac.id/bitstream/handle/123456789/56585/18523145.pdf?sequence=1&isAllowed=y>
- Arifin, S., Sendari, S., & Zaini, I. A. E. (2023). Perancangan Model Pergerakan Mobile Robot dengan Metode Deep Q Learning. *Jurnal Edukasi dan Penelitian Informatika (JEPIN)*, 9(3), 401. <https://doi.org/10.26418/jp.v9i3.64541>
- Daqiqil, I. (2021). *Machine Learning Teori, Studi Kasus dan Implementasi Menggunakan Python* (1 ed.). Badan Penerbit Universitas Riau.
- Goldberg, D. E., & Holland, J. H. (1988). *Genetic Algorithms and Machine Learning*. 95–99.
- Mehmood, S. (2024). *Reinforcement Learning for Automated Systems: Review of Concepts and Implementations*. 7.